



თბილისის თავისუფალი უნივერსიტეტი

მათემატიკის, კომპიუტერული მეცნიერებისა და ინჟინერიის
სკოლა (MACS[E])

კომპიუტერული მეცნიერებისა და მათემატიკის პროგრამა

ილია გოგოჭური

საბაკალავრო პროექტი

“Read Intent”

თბილისი

2026

ანოტაცია

“Read Intent” არის სტატიების და სხვა ტიპის ტექსტური მასალის კითხვისა და მოსმენისთვის განკუთვნილი აპლიკაცია.

აპლიკაცია მომხმარებლებს აძლევს საშუალებას შეინახონ სასურველი სტატიები პირდაპირ აპლიკაციიდან URL-ით ან ვებ ბრაუზერის Extension-ის გამოყენებით.

შენახული სტატიები მუშავდება, ინმინდება ყურადღების გამფანტავი ელემენტებისგან (რეკლამები, რეკომენდაციები, შემოთავაზებები არასტანდარტული ფორმატირება) და სინქრონდება მობილურ ან დესკტოპ აპლიკაციასთან. რის შემდგომაც, სტატიის წაკითხვა და მოსმენა შესაძლებელია ინტერნეტის გარეშეც “Offline” ფორმატში.

პროექტის მოტივაცია მოდის იქიდან, რომ ინტერნეტი და სოციალური მედიები უფრო და უფრო “უარესდება”. ალგორითმული რეკომენდაციების სისტემა ოპტიმიზირებულია მაქსიმალურად მიიტაცოს მომხმარებლის ყურადღება. “ქონტენტი”, რომელიც პოპულარული ხდება, არაა აუცილებელი იყოს რეალობასთან შესაბამისი, არათუ რაიმე ღირებულების შემცველი. ხოლო, ღირებულების მატარებელი ინტერნეტ-მასალა ხშირად იკარგება ამ ინფორმაციულ ქაოსში. ამას გენერაციული LLM სერვისებისა და ხელსაწყოების პოპულარობამ განსაკუთრებით შეუწყო ხელი. ევროპოლის რეპორტით ინტერნეტში განთავსებული მასალების 90%, 2026 წლისთვის იქნება “ხელოვნური ინტელექტის” მიერ დაგენერირებული. (EPRS, 2025)

“Read Intent” მომხმარებელს აძლევს შესაძლებლობა მარტივად შეინახოს მისთვის სასურველი და საინტერესო ტიპის “ქონტენტი”, რომელიც ასევე მარტივად იქნება ხელმისაწვდომი სასურველ ფორმატში. იმის ნაცვლად, რომ სოციალური მედიების ალგორითმულ რეკომენდაციებზე იყოს დამოკიდებული.

ვინაიდან ბოლო წლებში, განსაკუთრებით პოსტ-კოვიდ პერიოდში აუდიო ფორმატის მედიის, მათ შორის აუდიო წიგნების და პოდკასტების,

პოპულარობა გაიზარდა, აპლიკაციის მნიშვნელოვანი ნაწილია სტატიების მოსმენის შესაძლებლობა. აღსანიშნავია, რომ აპლიკაცია სტატიის რეალისტურთან მიახლოებულ აუდიო ფორმატს ასინთეზებს რეალურ დროში, უშუალოდ მომხმარებლის მონყობილობაზე. რაც ამცირებს ინფრასტრუქტურული რესურსების საჭიროებას მსგავსი სისტემების ოპერირებისას, რაც როგორც წესი წარმოადგენს ყველაზე დიდ ოპერაციულ ხარჯს.

სისტემის შემადგენელი კომპონენტებია: მობილური და დესკტოპ აპლიკაცია, სტატიების წასაკითხად და მოსასმენად; ვებ ბრაუზერების ექსტენშენი, სტატიების მარტივად შესანახად; და ქლაუდზე გაშვებული მიკრო-სერვისები - აუთენტიკაციისთვის, სტატიების დამუშავებისთვის, შენახვისთვის და სინქრონიზაციისთვის.

Abstract

“Read Intent” is a read-it-later application for reading and listening to text-based web content, primarily articles. Using the application, users can save articles from the app via URL or using the browser extension.

The saved articles are cleaned and sanitized by the system, removing unnecessary distractions such as ads, recommendations, offers, and non-standard formatting. The cleaned-up content is then downloaded to the app, giving you the ability to read or listen to your preferred content even when you are offline.

Motivation for the project comes from the fact that the internet as a whole, and especially social media, is getting “worse”. Algorithmic content recommendation systems are optimized to steal your attention as much as possible. Which, in turn, causes the promoted/popular content to be sensational, low-quality, and often even fake. At the same time, the content with any substance is often lost in informational chaos. The mass adoption of the generative LLM services and tools has further exacerbated the situation. Europol forecasts that 90% of online content will be synthetically generated by the end of 2026. (EPRS, 2025)

“Read Intent” lets users save content they want to consume and access it anywhere, in a format they desire, with minimal friction, rather than being fed by the algorithm.

In recent years, especially post-COVID, audio-style content has become highly popular: 40% of the US adults listen to podcasts weekly (up from 17% in 2018), and Audible, the largest digital audiobook store, has reached an all-time high revenue. Due to this trend, a central part of the app is the ability to turn articles into an audio, podcast-like experience. The application can synthesize real-time, realistic audio on the user’s device. This, in turn, drastically reduces the infrastructure costs required to run such systems, which is often the highest category in infrastructure op-ex.

The main components of the “Read Intent” platform: Mobile and desktop applications, for listening to and reading the articles; the browser extension to make saving articles simple; and the cloud micro-services - handling authentication, article processing, storage, and synchronization.

Table of contents

ანოტაცია	I
Abstract	II
Table of contents	VI
1. Introduction	1
2. Main Content	2
2.1 Current Market and Demand	2
2.2 Existing Solutions	3
2.3 Novelty and Challenges	4
2.4 Technical Decisions and Implementation	5
2.4.1 System Components and Technology Stack Decisions	6
2.4.2 Supporting Components and Infrastructure	8
2.4.3 Challenges and Approaches	10
2.5 Results and Conclusion	11
Citations and References	13
Appendix	15

1. Introduction

The internet has gotten worse over the last few years, and the trend suggests it will only get worse from here. The word of the year in 2025 has been "Slop" (Merriam-Webster, 2025). Europol reports (EPRS, 2025) estimated that up to 90% online contents could be generated synthetically by the end of 2026. And social media algorithms further incentivize and amplify the generation of such low-quality content, since the only relevant metric is engagement. Social media is pushing sensational, low-quality, and often fake content, and the content with any substance often gets lost in the informational chaos.

Oxford's Word of the Year 2024 has been "Brain rot" (Oxford University Press, 2024) - "refers to the cognitive decline and mental exhaustion experienced by individuals, particularly adolescents and young adults, due to excessive exposure to low-quality online materials, especially on social media" The research shows it "leads to emotional desensitization, cognitive overload, and a negative self-concept" (Yousef et al., 2025, #). Habitual consumption of negative news ("doomscrolling") is associated with elevated levels of existential dread and anxiety (Shabahang, 2024, #). And making healthy choices is difficult when instant gratification is one tap away and quality content requires active hunting.

People are starting to push back. Studies from multiple sources report an increase in digital burnout. Although the numbers vary significantly, the trend is increasing. A study in UK by Deloitte (Deloitte, 2025) - Illustrates the GenZs active disdain of social media, with one of the points being - "One in five (20%) consumers have deleted a social media app in the past 12 months, rising to one in three (29%) among Gen Zs;" And the "chronically offline" movement is gaining traction - 50% of 2,000 surveyed Americans reported intentionally make time to disconnect, with Gen Z and millennials leading at 63% and 57% respectively (Talker Research, 2026). However, most of the time, there is little to no way to replace this habit with something positive that offers the same level of (low) friction.

Audio content consumption is rising post-COVID. The Audible revenue is at an all-time high. In the US, 45% of adults now listen to podcasts weekly, up from just 17% in 2018 (Edison Research, 2026). This shows demand for audio-form

media content, especially during commuting and other screen-free moments.

The three trends above indicate an opening for a product that allows intentional consumption of high-quality content in an audio form. And articles from trusted sources and authors are the best way to acquire condensed knowledge from people who have spent their time researching and thinking about issues in fields you care about. “Read Intent” is based on the idea that this content should require minimal effort to find, save, and consume.

2. Main Content

2.1 Current Market and Demand

Mozilla shut down Pocket (Mozilla, 2025) in 2025 despite it having more than 3 billion saved pieces of content and 10 million active users in just 2017. In their announcement, they have mentioned strategic shift - “channeling our resources into projects that better match browsing habits today”, with a stronger focus on AI and browser-native features.

The shutdown has caused an outcry from the user community across multiple social media platforms. Users expressed frustration at losing the product they used daily and began searching for alternatives. If we follow discussions at the time (July 2025), there were almost no alternatives that filled the same niche. Although some newer products have been introduced since then, none has taken the same place of being a simple, listening-focused, read-it-later app, and the gap in the market still exists. Although I have not conducted formal market experiments.

None of the existing alternatives satisfies all the criteria for a simple, focused read-it-later app. Some have overly complex interfaces, others are bloated with features that distract you from the main goal of consuming content, yet others are too expensive. To save and listen to articles, you should not need to learn and adapt to an entirely new system. On top of that, existing alternatives are aggressively pushing AI features (summarization, chatbots, auto-tagging), and there is a growing

segment of users who want their tools without AI. This market is underserved currently.

2.2 Existing Solutions

The closest existing solutions in the read-it-later space are:

Instapaper - The most similar product to what Read Internt aims to be. It has all the features, but most of them are behind a paywall. It has a listening feature, but audio generation is slow and not real-time. The application has existed since 2008, and the user interface has not been modernized in over a decade, making it nonergonomic to use. Still, it has 500K+ users. Revenue is not public.

Matter - Well-regarded app, popular in professional circles. Matter has a polished and ergonomic interface. It is deliberately focused on perfecting the native iOS performance and UI and has indicated multiple times that they do not plan on expanding to multiple platforms. They have close to 100k users with revenue close to \$2M in 2025.

Speechify - A general-purpose application for converting multiple types of text-based media to audio, with a wide variety of celebrity voices. Speechify hardly classifies as a read-it-later app, and has no streamlined flow for saving article content. However, their success is encouraging - having 50M+ users and \$17.6M ARR.

Readwise (Reader) - Primarily a research and note-taking tool that works on many content types. The discovery and consumption of articles isn't the main focus, and they have a steeper learning curve. At around \$200 per year, the app is not worth it for most users. They are rumored to have 100K+ users.

Overall, none of the solutions above combine a simple, distraction-free reading experience with a primary focus on listening. These are not new products. Instapaper has existed since 2008; they have not innovated in a long time and pushed out features in a rush after the Pocket shutdown. Matter has deliberately focused on the IOS experience. Speechify and Readwise solve slightly different problems.

The feature gap is there because the existing solutions focus on different niches, and if they pivot their products more toward the "Pocket problem," that will only be an indication that the market is there.

Disclaimer: Note that the listed companies/products are not public, and the numbers are based on various pieces of information and observations, which likely reflect reality but may be inaccurate.

2.3 Novelty and Challenges

Recent advancements in deep learning, specifically transformer architectures and speech synthesis, allowed for the creation of incredibly small, realistic-sounding text-to-speech (TTS) models. One such model is Kokoro 82M (Hexgrad, n.d.) - Kokoro's architecture is based on 2 papers (**StyleTTS 2** (Li et al., 2023, #) **and ISTFTNet** (Kaneko et al., 2022, #)). Oversimplification of how this was possible - Kokoro has separate precomputed style vectors that can be loaded on demand, and no audio upsampling encoder neural network; instead, it predicts Short-Time Fourier Transform (STFT) coefficients per-frame and uses fixed inverse STFT to get the PCM audio bytes.

Kokoro's models are small enough to run easily on mobile devices using the ONNX runtime, which allows us to generate real-time on-device audio, with multiple different voice styles - allowing us to generate audio without the overhead of the expensive server infrastructure with dedicated GPUs.

There are several challenges that I have identified at the start of the project:

- Kokoro TTS has two main components. The repository itself is implemented in Python, which, by default, cannot run on Android devices, and is responsible for transforming pure text content into International Phonetic Alphabet (IPA) phonemes. And the weights are provided separately, taking encoded phoneme chunks as input and outputting PCM audio.
- PCM audio is by default in wave form, and WAV format doesn't natively support streaming, let alone real-time audio handling. All of this makes it tricky to support a proper audio player interactions.
- Parsing web content into pure text form, without losing data or making it noisy, is difficult.

2.4 Technical Decisions and Implementation

From the user's perspective, the **main user flow** for the application is quite simple - they submit an article for processing via the UI or the extension; the system scrapes the article and cleans it, generates phonemes for it, and sends an event to the UI when the article is ready; the user can see all of their saved articles in an inbox list and can interact with them; clicking the article opens the article reader and the attached player with real-time audio generation and sentence highlighting.

Below is a detailed breakdown of how all of that is accomplished and of every major component of the system involved.

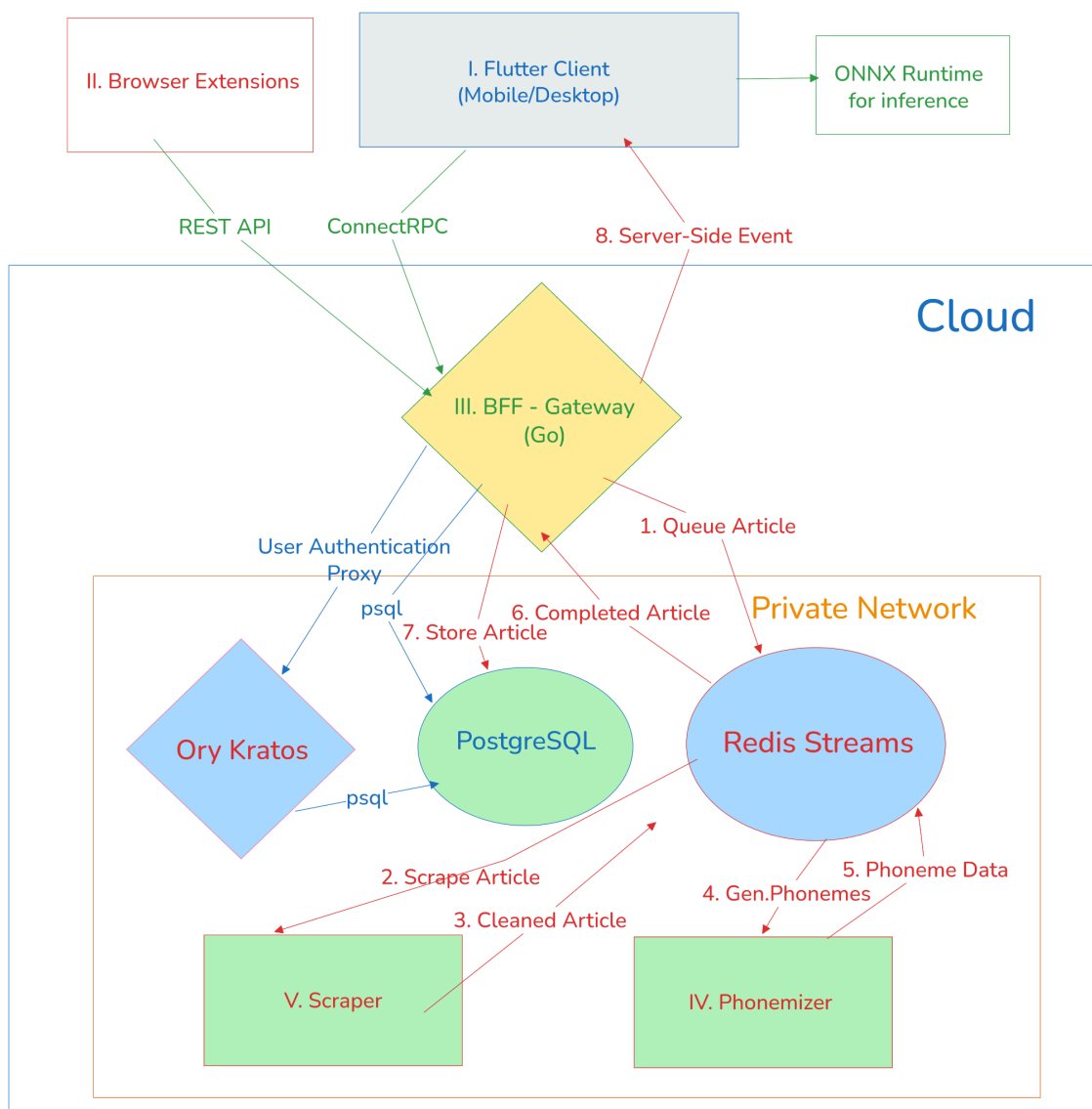


Figure 1. Overview of "Read Intent" architecture, components, and a simplistic overview of the event flow for the basic case.

2.4.1 System Components and Technology Stack Decisions

“Read Intent” is composed of the following components. I will provide an overview of each below (Figure 1). For a deeper dive, consult the accompanying GitHub repository and additional technical documentation there.

- I. **UI** - Cross-platform application written on **Flutter**
- II. **Browser Extension** - Written using manifest V3 (JS)
- III. **BFF** (Backend For Frontend) - The main API service and a modular monolith written in **Go**, acting as a primary gateway for the outside internet to interact with the backend services.
- IV. **Phonemizer** - **Python** service, responsible for generating phonemes based on the pure text extracted.
- V. **Scraper** - **Python** service, responsible for fetching, parsing, and cleaning the articles.

User Interface - framework should have satisfied the following requirements: I wanted it to be cross-platform, have a decent performance, and play well with the FFI bindings for the ONNX runtime and audio encoding libraries. The choice came down to React Native and Flutter, and **Flutter (Dart)** was chosen mostly because of its better performance and my experience with it. This UI is the primary interaction interface for the user. This is where the audio is generated from pre-generated phonemes for the target article.

Extension - There wasn't much of a choice; the manifest V3 is the current standard for cross-browser extensions. This browser works on Firefox and Chromium (and its derivatives). The extension has a Device Authentication Grant RFC8628. Basically, displays a code you can enter in the app to authenticate. And the main function of the extension is to save the current page or a custom URL to the articles backend.

Backend For Frontend - **Go** is a simple and performant language, and I am most comfortable writing services with it - this was an easy technology choice. This service acts as the main API gateway for the rest of the system. We are exposing a REST API for a health check endpoint and the extension consumption. We are also exposing the **ConnectRPC** HTTP service on the same server address for the Flutter UI.

Another function of the **BFF** is **authentication**, and it provides two paths for authentication. Primary authentication is done with the ConnectRPC endpoint, using email and password or Google social authentication. The endpoint issues a session token after authentication, and the entire authentication flow and management of OAuth2 is handled by the self-hosted instance of **Ory Kratos**. In this authentication branch, the BFF acts as a wrapper and authentication middleware. The second authentication method is an implementation of the "OAuth 2.0 Device Authorization Grant" ([RFC8628](#)). A summary of how this works: BFF issues a short code and unique device identifier when requested via the REST API; the short code is displayed by the browser extension; an already authenticated user (done through the primary way) can then enter the displayed code in the app and claim ownership of the device. Once the ownership of the device is claimed, we issue a signed JWT for the extension and grant the bearer of the token a limited set of authenticated actions (only allowing them to save articles).

The architecture of **BFF** is technically a modular monolith, with sub-components broken down into self-contained vertical slices: Articles service, Extension service, and Authentication service. The slices are assembled using Go interfaces and can be easily split apart if required. Putting each of the behind a different process with gRPC connections is trivial work. Each slice uses a simplified version of the clean architecture, with decoupled “ports” and “adapters” to define the interaction interface with the outside world. For instance, the Articles service and the Authentication service have the ConnectRPC port defined in front of the service. While the Extension service has a simple HTTP “port” for providing an HTTP REST interface.

Phonemizer - I have mentioned before that the Kokoro TTS source is written in Python, which means we will not be able to easily make it portable for a cross-platform application. The source code for Kokoro processes text and outputs phonemes; it then runs the pipeline with the model weights and outputs audio bytes. The second part of the process can be easily adapted for the Flutter application by using Kokoro models and style vectors specifically designed for the ONNX runtime, and running them on the ONNX runtime through FFI wrappers (such as [onnxruntime_v2](#) for Dart). The first part of the Kokoro pipeline, the phoneme generation part, is specifically designed to work well with the models, so we would like to use that directly.

Fortunately, after diving into the source code, there is a way to use the model only for the phoneme generation part. And the solution was to create a small **Python microservice** that ingests pure text of the article and outputs processed, chunked, and tokenized phoneme data. The Phonemizer service has no direct I/O; it is event-driven, taking events designated for phonemizer from Redis Streams and returning the processed data. The process acknowledges events that have been successfully completed or that aren't correctly formed. It is also implementing a retry for the events that haven't been acknowledged yet for some time.

Scraper - Most web scraping and content processing libraries are either in Javascript or Python. There have been a few candidates from each, but further research of the performance and ergonomics revealed the candidate I like the best - **Trafilatura**. Trafilatura is a **Python** library that can fetch the content via the URL, extract metadata, such as main image, author, date, category, etc., and extract the main content of the page, including relevant elements - tables, images, graphs, and code segments. The library is also easily extensible, which makes it easier to bolt on additional article transformations for our use case. The I/O for the **Scraper** is essentially done the same way as for the Phonemizer - using Redis Streams events. One additional note, that is also true for the Phonemizer: the event processing is done on an at-least-once basis. There are a few flows in which it is possible to process articles more than once within each service. As long as each service's output is correct, multiple deliveries don't break anything.

This specific microservice split was mainly due to each component having separate technology/stack needs. Furthermore, the split between the Scraper and the Phonemizer services is due to highly differing characteristics of computational and network resources for each. If we ever need to scale them individually, we would move the phonemizer to a server with a GPU, and Scraper might need to be distributed across different servers to avoid hammering the hosts of the articles from the same IP.

2.4.2 Supporting Components and Infrastructure

The technology stack is highly focused on simplicity, availability, and cost-effectiveness. Where possible, the stack choice includes open-source, source-available, and self-hostable software.

Database - PostgreSQL is the industry-standard open-source SQL database. “Read Intent” uses a self-hosted, containerized PostgreSQL instance with a persistent storage volume that is backed up regularly.

Event Delivery - Redis Streams (8.0+) is an ultra-low-latency, self-hostable event-sourcing platform. The choice for event sourcing was either Redis, which is free and open-source under the AGPLv3 license, or a commercial platform such as Kafka. Since there was essentially no need for a distributed event log, Redis was the best choice. A containerized Redis instance with WAL is self-hosted on the same machine and shares the same backed-up volume as PostgreSQL.

Authentication Provider - Ory Kratos is “an API first identity and user management system.” There aren’t many battle-tested, open-source, and yet simple enough OAuth2.0 authentication providers out there. The few that are good are either managed and require payment after a threshold is reached or over-engineered and designed for multi-tenant systems. We are also running the Kratos instance inside a Docker container. The instance uses the same PostgreSQL database instance for identity persistence.

Reverse Proxy - Caddy is a simple and easy-to-configure reverse proxy that sits in front of the BFF.

Container Orchestration - Docker Compose is the easiest way to orchestrate containers on a single machine.

Infrastructure Setup - OpenTofu is one of the leading open-source IaC software. OpenTofu is under the Linux Foundation and a member of CNCF. And since I am a maintainer, it was a straightforward choice.

The entire system currently runs on a single machine in **Hetzner Cloud**. Provisioning the server and all infrastructure components can be done with a single command: `tofu apply`.

Communication Layer and Schema Generation - ConnectRPC is a popular alternative to gRPC, used for definitions of application public interfaces and type/schema contracts in a fast, type-safe, and easily backward-compatible way with protocol buffers. Auto-generation of client/server code for UI and services makes development and contract changes much simpler, especially in multilingual programming projects. “Read Intent” uses this layer, coupled with simple Makefile scripts, to generate and distribute contract changes from a single source-of-truth Protobuf file.

“Read Intent” is versioning every component of the system in a single Git repository. This is also known as “monorepo.” Initially, I was thinking of creating separate repositories for each component. However, after trying this approach, managing multiple repositories and versioning them separately became too cumbersome. On top of that, the shared contract generation for ConnectRPC was easier in a monorepo environment.

CI/CD - GitHub and DockerHub - For continuous integration and continuous delivery, I have set up GitHub workflows for: automated testing on development and production environments; building Docker images and pushing them to DockerHub; deploying the newly built images on the server using SSH.

To avoid downtime during deployment, I have implemented a custom blue-green deployment for the BBF service. Starting an instance with the latest Docker image, ensuring it is healthy and up and running, and then switching traffic to it using Caddy.

2.4.3 Challenges and Approaches

When developing “Read Intent,” I started with proof of concepts (PoCs) for each component to avoid hitting big challenges later in the development process.

At first, I started experimenting with TTS generation pipelines. Getting it to work reliably was a challenge. There are a few steps before we start generating audio. Firstly, we need the model weights and style vectors, which can be several hundred megabytes in size. Some devices will not support certain quantization and precision levels due to hardware limitations and performance overhead, requiring us to try another version of the model. Hence, bundling all the weights from all available models with the application installer wasn’t an option, since it would make the installer unnecessarily heavy. I opted for downloading models at initial application startup, with a fallback.

Another issue was UI lag caused by high demand on the application's main thread for model loading and inference. Fortunately, the ONNX runtime library supported running inference in a separate isolate, and model loading was easy to move into a separate Dart isolate.

The final challenge for the audio generation pipeline was real-time audio streaming. Generated PCM bytes are easy to encode and convert directly into the WAV audio format without an additional encoder. But the WAV format doesn’t

natively support unknown sizes and streaming. I have tried several workarounds, including starting a local HTTP server to stream audio, but none worked. Finally, I ended up using the MP3 audio format with libmp3lame FFI for encoding the PCM bytes into a growing MP3 file and playing that with a custom audio player.

The next PoC was the scraper component. I initially assumed, based on my previous web scraping experience, that I would need to involve a headless browser in order to properly render the DOM on JavaScript-heavy pages. After developing and testing the base version of the scraper service, virtually every blog and article website worked well with the basic Trafilatura “fetch” component. Which led me to not needing to implement the headless browser version of the content fetching at all. This, in turn, made the server much lighter and more performant than it would be with a bundled browser.

Another issue I had was the lack of good ways to test the authentication service that wraps the Kratos API. The implementation was tightly coupled to complex Kratos types, which prevented me from easily mocking it. And even if I mocked it, the complex interactions would be lost during testing, and iteration would take a long time. I needed to run tests against the real Kratos instance running, which by default isn’t scalable. I have decided to use an awesome library for testing called Testcontainers. Testcontainers allowed me to run real Docker containers during unit and integration tests, and test against the actual dependencies such as Kratos.

I have also used Testcontainers for other tests in Python and Go services. For instance, using the real PostgreSQL instance is a great way to reliably test SQL repository implementations. Without such tests, we could use a psql-incompatible command and never know it until we run it against the real database. This approach adds overhead for each test. However, for stricter guarantees, the tradeoff is worth it. The overhead can be further reduced by reusing already started containers and using tmpfs mounts for data volumes, which use fast RAM as a storage device - minimizing database migration and execution times.

A few quality article websites are gated behind a paywall. If you have paid for the service or subscribed to a specific author, you are allowed to read their publications. I needed a way to allow users who can access such content to view it in the application. After trying a few workarounds, the easiest and safest way to do it was only to allow saving such pages with an authenticated browser session using

the extension. The extension copies the existing rendered DOM, serializes it, and sends it with a parsing request. That way, we never need to fetch articles behind a paywall; we use already-rendered state to parse and extract the information instead.

2.5 Results and Conclusion

The “Read Intent” application has all the main features I initially intended and scoped for the project. Additionally, I have managed to make the on-the-fly playback speed change work and have added a sentence highlighting sync with the audio playback. Unfortunately, time has not allowed me to include any other extras from the list (provided in the plan).

The UI application supports Mobile and Desktop platforms. I have tested Android and Linux environments, and it works without any issues.

The distribution channels for this application - I have submitted the app to the Android Play Store with a Google Developer account under the name “Read Intent”. The app is approved for internal testing, but until I gather enough testers to meet their policy requirements, I unfortunately cannot make it public. I have published the extension for Mozilla Firefox [<https://addons.mozilla.org/en-US/firefox/addon/readintent/>] and intend to submit it to Google Chrome as well. I have also acquired the domain “readintent.app”, although “Read Intent” doesn’t have a home page yet; the subdomains are used to bind DNS to the Hezner servers.

The current version of the application has the following main features from the user’s perspective:

- Email-Password authentication
- Google Sign In *
- Offline-first mode, requiring no connection
- Article saving from the UI and the browser extension
- Articles lists - Inbox/Favorite/Archive
- Article CRUD
- Article reader with
 - Formatted article
 - Full audio player with real-time audio generation

- Sentence highlighting synced with playback
- Mini player version on the screens outside the article reader to allow free navigation.
- Playback speed selection
- Preference and position persistence
- Voice style selector (with 20+ available voices)
- Font size scaling for accessibility
- Browser extension with device code authentication
- Dark/Light mode themes

Google Sign In worked until the last few days. Google paused the app's identity service and required additional compliance work from me. Which might not be sorted in time, this is reviewed.

As for whether “Read Intent” will stay free or be monetized in the future, I haven't decided yet. A lot depends on the current market and whether they meet the forecasted demand. Or in other words, if “Read Intent” will have users.

In the end, I have worked with and gained hands-on experience in many technologies I previously understood only in theory or hadn't worked with extensively. Especially valuable experience has been linking everything end-to-end, the full stack from infrastructure management and DevOps pipelines to event-driven services to mobile user interfaces. Working in the current market rarely allows for such diverse experience and an end-to-end understanding of systems. And with all that, I have built an application that helps me form better habits, find interesting and valuable things to learn, and to read with more intent.

Citations and References

Deloitte. (2025, 06 05). *Gen Zs favour social media ban for under 16s as digital fatigue hits*.

<https://www.deloitte.com/uk/en/about/press-room/gen-zs-favour-social-media-ban-for-under-16s-as-digital-fatigue-hits.html>

Edison Research. (2026, 03 12). *The Infinite Dial® 2026*.

<https://www.edisonresearch.com/the-infinite-dial-2026/>

EPRS. (2025, 07 01). *Children and deepfakes*. European Parliament. Retrieved June 11, 2026, from

[https://www.europarl.europa.eu/RegData/etudes/BRIE/2025/775855/EPRS_BRI\(2025\)775855_EN.pdf](https://www.europarl.europa.eu/RegData/etudes/BRIE/2025/775855/EPRS_BRI(2025)775855_EN.pdf)

Hexgrad. (n.d.). *hexgrad/Kokoro-82M*. Github. Retrieved June 11, 2026, from

<https://github.com/hexgrad/kokoro>

Kaneko, T., Tanaka, K., Kameoka, H., & Seki, S. (2022). iSTFTNet: Fast and Lightweight Mel-Spectrogram Vocoder Incorporating Inverse Short-Time Fourier Transform. *ICASSP, 2022*. doi.org/10.48550/arXiv.2203.02395

Li, Y. A., Han, C., & Raghavan, V. S. (2023). StyleTTS 2: Towards Human-Level Text-to-Speech through Style Diffusion and Adversarial Training with Large Speech Language Models. *Advances in Neural Information Processing Systems*, 36.

doi.org/10.48550/arXiv.2306.07691

Merriam-Webster. (2025, 12 15). *Word of the Year 2025 | Slop*. Merriam-Webster.

Retrieved June 11, 2026, from

<https://www.merriam-webster.com/wordplay/word-of-the-year>

Mozilla. (2025, May 22). *Investing in what moves the internet forward*. The Mozilla Blog. Retrieved June 11, 2026, from

<https://blog.mozilla.org/en/mozilla/building-whats-next/>

Oxford University Press. (2024, December 2). *'Brain rot' named Oxford Word of the Year 2024*. Oxford University Press. Retrieved June 11, 2026, from <https://corp.oup.com/news/brain-rot-named-oxford-word-of-the-year-2024/>

Shabahang, E. A. (2024). Doomscrolling evokes existential anxiety and fosters pessimism about human nature? Evidence from Iran and the United States. *Computers in Human Behavior Reports*, 15. doi.org/10.1016/j.chbr.2024.100438

Talker Research. (2026, 01 8). *Survey reveals intentional digital disconnection growing among Americans*. talkerresearch.com. <https://talkerresearch.com/survey-reveals-intentional-digital-disconnection-growing-among-americans/>

Warade, P. (2026, 07 03). *Text to Speech Software Market Size, Share & Trends Analysis Report By Component (Solution, Services), By Deployment Mode (Cloud, On-Premise), By Organization Size (SMEs, Large Enterprise), By Industry Vertical*. <https://straitsresearch.com/report/text-to-speech-software-market>

Yousef et al. (2025, 02 09). Demystifying the new dilemma of brain rot in the digital era: A review. *Brain Sciences*, 15(3). doi.org/10.3390/brainsci15030283

Appendix

A public GitHub repository containing the complete source code for this project - <https://github.com/Gogotchuri/readintent>

Google Drive - Open to FREEUNI users, containing screenshots, presentation, demos, and miscellaneous items - <https://drive.google.com/drive/folders/15EtEugddx8DnZPPnIDIYeGcu7Wowx3o7?usp=sharing>